Software Engineering Lab

Lab 27

Database Constraints: Benteng Terakhir Integritas Data

Mencegah Race Condition, Menjaga Invarian Skema,
dan Menolak Duplikasi Atomik di Go



Lukman / lukman-ss

Validasi Aplikasi Runtuh di Bawah Beban Konkuren

Pola check-then-act di memori aplikasi membuka jendela tabrakan data duplikat.

DAMPAK KEGAGALAN VALIDASI HANYA DI APLIKASI

Race Condition Registrasi

Dua request membaca 'email belum ada' secara bersamaan, lalu keduanya melakukan INSERT email identik.

DAMPAK KEGAGALAN VALIDASI HANYA DI APLIKASI

Deduplikasi Pasca-Fakta Mahal

Membersihkan data duplikat di produksi membutuhkan rekonsiliasi manual, migration rumit, dan downtime.

DAMPAK KEGAGALAN VALIDASI HANYA DI APLIKASI

Bukan Berarti Bug Kode

Kegagalan ini bukan berarti logika if/else salah, melainkan batasan fundamental arsitektur tanpa penguncian atomik.

Pola check-then-act yang memeriksa ketersediaan data via SELECT sebelum melakukan INSERT selalu tampak berhasil pada pengujian sekuensial. Namun di bawah beban konkuren, dua thread membaca kondisi yang sama sebelum salah satunya menulis. Kegagalan ini bukan berarti kode if-check rusak, melainkan akibat ketiadaan isolasi atomik pada titik penyimpanan.

Application-level checks cannot bridge the race window between read and write operations under concurrency.

Mengapa Review Manual dan Pengecekan Memori Gagal?

Pemeriksaan memori tanpa proteksi engine database tidak mampu menahan beban paralel.

KELEMAHAN PENDEKATAN VALIDASI NAIF

Interleaving CPU Scheduling

Jeda milidetik antara query SELECT dan INSERT dieksploitasi oleh OS scheduler untuk menyisipkan operasi lain.

KELEMAHAN PENDEKATAN VALIDASI NAIF

Unit Test Sekuensial Menipu

Test standar tanpa konkurensi goroutine selalu pass 100%, menciptakan ilusi keamanan palsu bagi tim engineer.

KELEMAHAN PENDEKATAN VALIDASI NAIF

Tidak Otomatis Terlindungi

Menjalankan validasi di microservice terpisah tidak otomatis menjamin keunikan tanpa koordinasi locking atomik.

Review manual sering melewatkan celah integritas karena kode validasi tampak sangat rapi dan logis di editor. Unit test sekuensial selalu memberikan status hijau karena tidak ada kompetisi thread. Menjalankan validasi di lapisan aplikasi tidak otomatis melindungi sistem jika engine penyimpanan tidak menegakkan kunci eksklusif pada saat penulisan.

Sequential testing creates an illusion of correctness that shatters under concurrent thread interleaving.

Model Mental Dua Lapis: Kolaborasi Aplikasi dan Database

Memisahkan peran kenyamanan pengguna (UX) dari jaminan kebenaran data mutlak.

USER EXPERIENCE (UX)

Lapis 1: Application Validation

Memberikan pesan error ramah, validasi format cepat, dan sanitasi input sebelum request menyentuh database.

LAST LINE OF TRUTH

Lapis 2: Database Constraints

Menjamin invarian skema secara deklaratif dan atomik dengan lock internal engine saat baris data ditulis.

SQLSTATE TAXONOMY

Jembatan: Error Mapping

Hanya pada jalur ini error database 23xxx diterjemahkan menjadi domain error terstruktur seperti HTTP 409 Conflict.

Arsitektur modern membagi tanggung jawab menjadi dua lapis: aplikasi melayani keramahan pesan pengguna, sedangkan database menjaga kebenaran data absolut. Validasi aplikasi menyaring input yang salah format dengan cepat. Hanya pada jalur ini, ketika terjadi tabrakan konkuren, database constraint menolak pelanggaran secara atomik dan mengembalikan kode SQLSTATE terstandar.

Application validation provides user-friendly guidance; database constraints provide inviolable invariant enforcement.

Urutan Evaluasi Constraint: Alur Validasi Internal Mesin

Bagaimana storage engine memproses aturan skema secara berurutan sebelum commit baris.

COLUMN SCOPE

1. NOT NULL (23502)

Memeriksa kekosongan nilai field wajib; gagal seketika jika kolom mandatori kosong.

PREDICATE SCOPE

2. CHECK (23514)

Mengevaluasi ekspresi boolean pada baris (misalnya age >= 18 atau status valid).

INDEX SCOPE

3. UNIQUE / PARTIAL (23505)

Memeriksa tabrakan nilai pada B-tree index aktif di bawah proteksi lock eksklusif.

RELATION SCOPE

4. FOREIGN KEY (23503)

Memverifikasi keberadaan ID induk pada tabel referensi sebelum commit data.

STORAGE COMMIT

5. ATOMIC COMMIT

Baris data dan entri index ditulis bersamaan; belum membuktikan engine multi-node di lab ini.

Storage engine mengevaluasi batasan data dalam urutan deterministik: NOT NULL diperiksa lebih dulu, disusul ekspresi boolean CHECK, lalu penelusuran UNIQUE index, dan terakhir FOREIGN KEY. Jika salah satu tahap gagal, operasi dibatalkan seketika tanpa ada baris data atau indeks yang tertulis. Lab ini memodelkan alur sekuensial tersebut secara atomik di dalam satu critical section.

Deterministic constraint pipelines reject violations early, guaranteeing zero partial writes upon schema conflicts.

Evaluasi Atomik Constraint di Bawah Mutex Lock

Memastikan tidak ada jeda antara pemeriksaan aturan dan penulisan baris data.

```
func (e *Engine) InsertUser(u model.User, usePartial bool) (model.User, error) {
    e.mu.Lock()
    defer e.mu.Unlock()
    if u.Email == "" { return model.User{}, dberr.NewNotNullViolation("users", "email", "users_email_not_null") }
    if u.Age < 18 { return model.User{}, dberr.NewCheckViolation("users", "users_age_check", "age >= 18") }
    if exists := e.emailIndex[u.Email]; exists {
        return model.User{}, dberr.NewUniqueViolation("users", "users_email_key", "duplicate email")
    }
    u.ID = e.userSeq.Add(1); e.users[u.ID] = u; e.emailIndex[u.Email] = u.ID
    return u, nil
}
```

Metode InsertUser menjalankan seluruh pengecekan di dalam satu blok mutex lock yang merepresentasikan ROW EXCLUSIVE lock pada PostgreSQL. Mengunci seluruh proses bukan berarti implementasi ini memakai latch halaman B-tree yang kompleks, melainkan memodelkan jaminan atomisitas agar tidak ada goroutine lain yang dapat menyisipkan email serupa.

Encapsulating validation and mutation within an atomic critical section eliminates check-then-act vulnerabilities.

internal/engine/engine.go:45–99

Kronologi Tabrakan: Dua Goroutine pada Validasi Naif

Langkah demi langkah runtuhnya integritas data pada UnsafeStore tanpa constraint.

FASE TERJADINYA DUPLIKASI DATA SECARA KONKUREN

1. Pengecekan Bersamaan

Goroutine A dan B sama-sama memindai tabel dan membaca bahwa email target belum terdaftar.

FASE TERJADINYA DUPLIKASI DATA SECARA KONKUREN

2. Jeda Penjadwalan CPU

Keduanya memasuki jeda scheduling (time.Sleep) yang membuka race window di memori aplikasi.

FASE TERJADINYA DUPLIKASI DATA SECARA KONKUREN

3. Penulisan Ganda Sukses

Goroutine A menulis row; Goroutine B langsung menyusul menulis row dengan email yang sama persis.

FASE TERJADINYA DUPLIKASI DATA SECARA KONKUREN

4. Integritas Runtuh

Hanya pada jalur ini tabel memuat 2 baris dengan email sama, merusak invarian keunikan sistem.

Ketika dua goroutine mengeksekusi UnsafeStore.RegisterUser secara paralel, keduanya sama-sama menemukan bahwa email belum terdaftar di database. Jeda scheduling CPU mengekspos race window sehingga kedua proses sama-sama melanjutkan ke langkah INSERT. Hanya pada jalur ini duplikasi data berhasil lolos ke penyimpanan, membuktikan bahwa pengecekan memori tanpa constraint tidak berdaya.

Uncoordinated concurrent reads before unconstrained writes guarantee permanent data corruption.

Matriks Taksonomi: Jenis Constraint vs Karakteristik

Membedakan cakupan validasi, kode SQLSTATE standar, dan mitigasi arsitektur.

Jenis Constraint	SQLSTATE	Cakupan Evaluasi	Pencegahan Kegagalan	HTTP Domain Mapping
NOT NULL	23502	Kolom Tunggal	Mencegah record kosong tak beridentitas	400 Bad Request
CHECK	23514	Kolom / Baris	Menolak nilai di luar batas bisnis logis	422 Unprocessable Entity
UNIQUE	23505	Antar Baris (Index)	Mencegah race condition registrasi ganda	409 Conflict
FOREIGN KEY	23503	Lintas Tabel	Mencegah orphan record (induk tidak ada)	400 Reference Error
PARTIAL UNIQUE	23505	Baris Bersyarat	Mengizinkan reuse email pasca soft-delete	409 Conflict

Tabel ini memetakan lima mekanisme constraint yang diuji di dalam lab beserta kode SQLSTATE kelas 23 standar PostgreSQL. Masing-masing mekanisme menangani dimensi integritas yang berbeda, mulai dari kelengkapan kolom hingga relasi lintas tabel. Menginspeksi kode SQLSTATE memungkinkan backend memetakan kegagalan teknis menjadi status HTTP yang semantik bagi klien.

Class 23 SQLSTATE error codes provide unambiguous machine-readable contracts for client error mapping.

Tiga Keputusan Kunci Merancang Integritas Skema

Panduan memilih mekanisme penegakan data yang tepat pada arsitektur produksi.

HIERARKI KEPUTUSAN PENEGAKAN INTEGRITAS DATA

1. Mandatori & Predikat Baris

Terapkan NOT NULL dan CHECK pada setiap kolom yang memiliki batasan domain eksplisit.

HIERARKI KEPUTUSAN PENEGAKAN INTEGRITAS DATA

2. Keunikan & Pola Penghapusan

Pilih UNIQUE biasa untuk entitas permanen, atau Partial Index jika sistem menerapkan soft-delete.

HIERARKI KEPUTUSAN PENEGAKAN INTEGRITAS DATA

3. Integritas Relasional & Index

Pasang FOREIGN KEY untuk mencegah orphan row; bukan berarti cukup, kolom relasi wajib di-index manual.

Desain skema yang tangguh dimulai dari mendefinisikan kolom wajib dengan NOT NULL dan batasan nilai dengan CHECK. Ketika mengelola entitas unik yang mendukung soft-delete, gunakan partial unique index agar email lama dapat didaftarkan kembali. Memasang foreign key bukan berarti pekerjaan selesai; kolom referensi tetap wajib dibuatkan indeks terpisah agar performa query tetap optimal.

Design schema invariants strictly at the column, table, and relational boundaries with appropriate index backing.

Verifikasi Eksekusi: Test Suite dan Stress Test 50 Goroutine

Output terminal membuktikan penolakan atomik SQLSTATE 23505 dan kelulusan race detector.

```
=== RUN   TestNotNullConstraints
--- PASS: TestNotNullConstraints (0.00s)
=== RUN   TestCheckConstraints
--- PASS: TestCheckConstraints (0.00s)
=== RUN   TestUniqueConstraint
--- PASS: TestUniqueConstraint (0.00s)
=== RUN   TestForeignKeyConstraint
--- PASS: TestForeignKeyConstraint (0.00s)
=== RUN   TestPartialUniqueIndex
--- PASS: TestPartialUniqueIndex (0.00s)
=== RUN   TestConcurrentRegistration_Safe_EnforcesUniqueness
--- PASS: TestConcurrentRegistration_Safe_EnforcesUniqueness (0.00s)
=== RUN   TestErrorClassification
--- PASS: TestErrorClassification (0.00s)
PASS: ok  labs/27-database-constraints/internal/store  1.330s (race: clean)
[DEMO] CONCURRENCY STRESS TEST: 50 GOROUTINES FOR SAME EMAIL
Results: Total=50 | Successful=1 | Rejected 23505=49 | Integrity Intact=true
```

Bukti eksekusi menunjukkan seluruh skenario pengujian lolos bersih tanpa peringatan dari Go race detector. Pada stress test demo dengan 50 goroutine yang menyerang email yang sama persis, storage engine berhasil meloloskan tepat 1 registrasi dan menolak 49 request lainnya dengan error 23505. Hal ini mengonfirmasi integritas data terjaga 100% di bawah beban paralel tinggi.

Stress testing confirms exactly one writer succeeds while all concurrent competitors receive atomic constraint rejections.

Batasan Bukti Lab: Yang Dibuktikan vs Belum

Menegaskan ruang lingkup simulasi in-memory terhadap fitur engine database produksi.

BATAS PEMBUKTIAN SISTEM LAB 27

YANG DIBUKTIKAN

Semantik SQLSTATE 23xxx, pencegahan race condition via atomic lock, dan siklus hidup partial index.

BATAS PEMBUKTIAN SISTEM LAB 27

YANG TIDAK DIBUKTIKAN

Constraint terdistribusi multi-node, isolasi SERIALIZABLE, operator GiST, dan B-tree latch riil.

BATAS PEMBUKTIAN SISTEM LAB 27

BATAS KLAIM EMPIRIS

Lab belum membuktikan teknik migrasi ALTER TABLE ADD CONSTRAINT NOT VALID pada tabel produksi raksasa.

Lab ini membuktikan semantik SQLSTATE, keefektifan atomic locking, dan mekanisme partial index menggunakan simulator in-memory Go murni. Pengujian ini belum membuktikan constraint pada database terdistribusi multi-node atau penggunaan operator eksklusi GiST. Batasan klaim ini penting agar engineer tidak menyamakan simulator lab dengan benchmark engine produksi penuh.

Delineate between educational memory simulations and production storage engine operational complexities.

Kompromi Rekayasa: Constraints vs Fleksibilitas Skema

Menimbang biaya operasional dan performa di balik perlindungan integritas absolut.

ANALISIS TRADE-OFF PENERAPAN DATABASE CONSTRAINTS

Overhead Tulis (Write Cost)

Setiap operasi INSERT/UPDATE wajib mengevaluasi index B-tree dan relasi FK, menambah sedikit latensi penulisan.

ANALISIS TRADE-OFF PENERAPAN DATABASE CONSTRAINTS

Friksi Migrasi Produksi

Menambah constraint pada tabel bervolume besar membutuhkan table lock eksklusif kecuali memakai pola validasi bertahap.

ANALISIS TRADE-OFF PENERAPAN DATABASE CONSTRAINTS

Tidak Otomatis Lambat

Penambahan index constraint tidak otomatis memperlambat sistem; overhead tulis terbayar lunas oleh integritas yang terjamin.

Menerapkan database constraints menambahkan sedikit biaya komputasi pada saat INSERT karena engine harus memeriksa B-tree index dan tabel induk. Selain itu, menambahkan constraint pada tabel miliaran baris di produksi memerlukan strategi khusus agar tidak mengunci tabel terlalu lama. Namun, overhead tersebut tidak otomatis memperlambat sistem jika dibandingkan biaya besar rekonsiliasi data rusak.

Accept minor write latency at the storage tier to permanently eradicate catastrophic data reconciliation costs.

Pipeline Validasi Konkuren: Goroutine vs Mutex Lock

Bagaimana 50 goroutine bersaing memperebutkan satu slot pendaftaran email unik.

TRAFFIC SURGE

1. 50 Goroutines Spawn

50 worker dijalankan serentak mencoba mendaftarkan email race-target@domain.com.

ATOMIC CRITICAL SECTION

2. Mutex Lock Serialization

Goroutine pertama memperoleh lock eksklusif; 49 worker lainnya mengantre tertib.

WINNER COMMIT

3. First Insert Success

Worker 1 lolos pengecekan, mencatatkan email ke tabel dan map index, lalu melepas lock.

ATOMIC REJECTION

4. 49 Abort SQLSTATE 23505

Worker 2–50 bergantian masuk lock, mendeteksi email sudah ada, dan langsung menerima error 23505.

Diagram ini memvisualisasikan penanganan 50 goroutine konkuren yang mencoba menulis email yang sama persis. Antrean mutex memaksa eksekusi berjalan serial pada saat penulisan, sehingga goroutine pemenang pertama berhasil mendaftarkan data. Seluruh goroutine berikutnya secara deterministik ditolak oleh pemeriksaan indeks unik, membuktikan bahwa isolasi atomik memotong habis celah race condition.

Atomic serialization at the storage layer transforms concurrent race conditions into deterministic rejections.

Transformasi: Dari UnsafeStore ke SafeStore

Perbedaan nyata antara validasi manual di aplikasi versus deklarasi constraint database.

PERBANDINGAN KETAHANAN DUA PENDEKATAN

Sebelum: UnsafeStore (App Check)

Mengandalkan for-loop di memori aplikasi; 20 goroutine menghasilkan >1 user duplikat di penyimpanan.

PERBANDINGAN KETAHANAN DUA PENDEKATAN

Sesudah: SafeStore (DB Constraint)

Mendelegasikan ke storage engine; 50 goroutine menghasilkan tepat 1 user dan 49 error SQLSTATE 23505.

PERBANDINGAN KETAHANAN DUA PENDEKATAN

Bukan Berarti Redundan

Mempertahankan validasi aplikasi bukan berarti pemborosan kode, melainkan filter awal bagi kenyamanan pengguna.

Sebelum menerapkan database constraints, UnsafeStore mengandalkan pemindaian memori yang mudah ditembus oleh 20 goroutine konkuren hingga menghasilkan baris duplikat. Setelah beralih ke SafeStore, delegasi ke engine database menjamin tepat satu pemenang dari 50 request paralel. Mempertahankan validasi aplikasi di depan bukan berarti redundan, melainkan jembatan penting untuk menyajikan pesan error yang informatif.

Shift data integrity enforcement from vulnerable application loops to unyielding storage engine constraints.

Verifikasi Test: Membuktikan Kebocoran Data pada UnsafeStore

Pengujian membuktikan bahwa pengecekan di memori gagal menahan 20 goroutine.

```
func TestConcurrentRegistration_Unsafe_SuffersRaceCondition(t *testing.T) {
    eng := engine.NewEngine(); s := store.NewUnsafeStore(eng)
    goroutines := 20; var wg sync.WaitGroup; wg.Add(goroutines)
    for i := 0; i < goroutines; i++ {
        go func(idx int) {
            defer wg.Done()
            _, _ = s.RegisterUser(ctx, model.User{Email: "unsafe@example.com", Username: fmt.Sprintf("u_%d", idx)})
        }(i)
    }
    wg.Wait()
    if eng.GetUsersCount() <= 1 { t.Errorf("expected race condition causing >1 user entries, got %d", eng.GetUsersCount()) }
}
```

Unit test ini membuktikan secara empiris bahwa pendekatan validasi di aplikasi gagal total di bawah konkurensi. Dua puluh goroutine yang mendaftarkan email yang sama menghasilkan lebih dari satu entri di tabel data. Kegagalan ini cakupannya mengikuti test lab, mengonfirmasi secara faktual bahwa sistem tanpa constraint penyimpanan pasti mengalami korupsi data saat menerima lonjakan request.

Empirical test assertions prove that application-only checks reliably permit duplicate writes under thread competition.

internal/store/store_test.go:210–239

Verifikasi Test: SafeStore Menjamin Tepat Satu Pemenang

20 goroutine paralel menghasilkan tepat 1 keberhasilan dan 19 penolakan atomik.

```
func TestConcurrentRegistration_Safe_EnforcesUniqueness(t *testing.T) {
    eng := engine.NewEngine(); s := store.NewSafeStore(eng)
    goroutines := 20; var wg sync.WaitGroup; wg.Add(goroutines)
    successCount, errorCount := 0, 0; var mu sync.Mutex
    for i := 0; i < goroutines; i++ {
        go func(idx int) {
            defer wg.Done()
            _, err := s.RegisterUser(ctx, model.User{Email: "safe@example.com", Username: fmt.Sprintf("u_%d", idx), Age: 25, Status: "active"})
            mu.Lock(); defer mu.Unlock()
            if err == nil { successCount++ } else { errorCount++ }
        }(i)
    }
    wg.Wait()
    if successCount != 1 || errorCount != 19 || eng.GetUsersCount() != 1 { t.Fatalf("integrity violated!") }
}
```

Pengujian ini membuktikan ketangguhan SafeStore yang mengandalkan constraint engine. Dari 20 goroutine yang dijalankan serentak, tepat satu pendaftaran yang berhasil ditulis ke penyimpanan, sementara 19 goroutine lainnya menerima penolakan. Hanya pada

Database constraints deterministically ensure a single commit winner and N-1 clean rejections under high concurrency.

internal/store/store_test.go:162-208

Solusi Soft-Delete: Partial Unique Index WHERE deleted_at IS NULL

Mengizinkan daur ulang email lama tanpa mengorbankan keunikan data pengguna aktif.

```
// PARTIAL UNIQUE: CREATE UNIQUE INDEX users_active_email_idx ON users (email) WHERE deleted_at IS NULL;
if usePartialUniqueIndex {
    if u.DeletedAt == nil {
        if existingID, exists := e.activeEmails[u.Email]; exists {
            return model.User{}, dberr.NewUniqueViolation("users", "users_active_email_idx", "duplicate active email")
        }
    }
}
// Saat soft delete: delete(e.activeEmails, u.Email) -> email langsung dapat didaftarkan kembali
// Baris dengan deleted_at != nil tidak diindeks, sehingga multi-duplikat soft-delete tetap valid.
```

Penerapan UNIQUE biasa pada tabel dengan soft-delete menimbulkan masalah: email pengguna yang sudah dihapus tidak akan pernah bisa didaftarkan kembali. Dengan Partial Unique Index bersyarat WHERE deleted_at IS NULL, hanya akun aktif yang diperiksa keunikannya. Pengguna yang sudah soft-delete dikeluarkan dari indeks, sehingga email yang sama dapat dipakai ulang oleh pendaftar baru.

Partial indexes scope uniqueness predicates exclusively to active records, resolving soft-delete reuse dilemmas.

internal/engine/engine.go:71–77

Mencegah Data Yatim Piatu dengan Foreign Key Constraint

Memastikan pesanan baru tidak dapat dibuat untuk pengguna yang tidak terdaftar.

```
func (e *Engine) InsertOrder(o model.Order) (model.Order, error) {
    e.mu.Lock(); defer e.mu.Unlock()
    if o.UserID == 0 { return model.Order{}, dberr.NewNotNullViolation("orders", "user_id", "orders_user_id_not_null") }
    if o.TotalCents <= 0 { return model.Order{}, dberr.NewCheckViolation("orders", "orders_total_cents_check", "total > 0") }
    // FOREIGN KEY: CONSTRAINT fk_orders_user FOREIGN KEY (user_id) REFERENCES users(id)
    if _, exists := e.users[o.UserID]; !exists {
        return model.Order{}, dberr.NewForeignKeyViolation("orders", "fk_orders_user", fmt.Sprintf("key (user_id)=(%d) not present", o.UserID))
    }
    o.ID = e.orderSeq.Add(1); e.orders[o.ID] = o; return o, nil
}
```

Fungsi InsertOrder mengevaluasi tiga batasan secara berurutan: NOT NULL pada UserID, CHECK pada TotalCents, dan verifikasi relasional FOREIGN KEY. Jika UserID bernilai 99999 yang tidak tercatat di tabel users, sistem mengembalikan SQLSTATE 23503. Mekanisme ini mencegah

Foreign keys enforce relational parentage at the storage boundary, eradicating orphaned dependent records.

internal/engine/engine.go:122–148

Inspeksi SQLSTATE: Mengubah Error Database Menjadi Domain Error

Hindari parsing teks error yang rapuh; periksa kode SQLSTATE Class 23 yang stabil.

```
func MapToDomainError(err error) error {
    var cErr *ConstraintError
    if errors.As(err, &cErr) {
        switch cErr.Code {
        case SQLStateUniqueViolation: // 23505
            return fmt.Errorf("conflict: resource with this unique attribute already exists (rule: %s)", cErr.ConstraintName)
        case SQLStateNotNullViolation: // 23502
            return fmt.Errorf("invalid input: mandatory field is missing (rule: %s)", cErr.ConstraintName)
        case SQLStateCheckViolation: // 23514
            return fmt.Errorf("validation failed: value outside boundary (rule: %s)", cErr.ConstraintName)
        case SQLStateForeignKeyViolation: // 23503
            return fmt.Errorf("reference error: referenced entity does not exist (rule: %s)", cErr.ConstraintName)
        }
    }
    return err
}
```

Banyak pengembang melakukan parsing string error seperti duplicate key yang sangat rapuh terhadap pergantian bahasa basis data. Kode SQLSTATE Class 23 bersifat universal dan terstandarisasi. Menggunakan MapToDomainError memungkinkan aplikasi mengubah kode mesin menjadi pesan domain yang bersih tanpa mengekspos struktur internal database ke publik.

Inspect stable SQLSTATE codes rather than brittle localized text strings for robust application error mapping.

Checklist Penerapan Database Constraints di Produksi

Langkah-langkah rekayasa wajib untuk menjamin integritas data sebelum deployment.

- ✓ Tandai seluruh kolom mandatori dengan NOT NULL, jangan hanya mengandalkan validasi form.
- ✓ Terapkan UNIQUE constraint pada atribut unik alih-alih melakukan SELECT sebelum INSERT.
- ✓ Gunakan Partial Unique Index (WHERE deleted_at IS NULL) untuk entitas yang mendukung soft-delete.
- ✓ Definisikan CHECK constraint untuk membatasi rentang nilai angka dan enum status kolom.
- ✓ Pasang indeks eksplisit pada kolom referencing FOREIGN KEY demi mencegah full table scan saat DELETE parent.
- ✓ Periksa kode SQLSTATE Class 23 (23502, 23505, 23514, 23503) pada error handler aplikasi.

Systematically verify declarative constraints, index backing, and structured error mapping across the persistence tier.

Anti-Pattern Umum dalam Menjaga Integritas Data

Kesalahan desain arsitektur yang sering merusak konsistensi data di sistem backend.

POLA BERBAHAYA YANG WAJIB DIHINDARI

Check-Then-Act di Aplikasi

Mengecek keberadaan record via SELECT di aplikasi lalu melakukan INSERT; pasti bocor di bawah beban konkuren.

POLA BERBAHAYA YANG WAJIB DIHINDARI

Menebak String Error

Melakukan regex matching pada teks error database yang dapat berubah sewaktu-waktu akibat pembaruan versi DB.

POLA BERBAHAYA YANG WAJIB DIHINDARI

Bukan Berarti Ter-Index

Membuat FOREIGN KEY bukan berarti kolom otomatis terindeks; ketiadaan indeks manual merusak performa query DELETE.

Anti-pattern paling fatal adalah merasa aman dengan pola SELECT sebelum INSERT pada aplikasi. Kesalahan lainnya mencakup parsing teks error alih-alih membaca SQLSTATE, serta berasumsi bahwa foreign key secara otomatis membuatkan indeks pada kolom anak. Mengabaikan aspek ini bukan berarti sistem rusak hari ini, melainkan menanam bom waktu performa di masa depan.

Avoid application-only uniqueness checks, error string parsing, and unindexed foreign key dependencies.

Reproduksi Mandiri di Mesin Lokal Menggunakan Go

Jalankan seluruh pengujian constraint, race condition, dan demo 50 worker tanpa dependensi eksternal.

```
cd labs/27-database-constraints
go test -v ./...    # menjalankan 7 unit test untuk NOT NULL, CHECK, UNIQUE, FK, & Partial Index
go test -race ./... # memvalidasi eksekusi konkuren bebas dari data race (PASS in 1.33s)
go run ./cmd/demo   # eksekusi demonstrasi CLI interaktif: SQLSTATE demo & 50-worker stress test
```

Seluruh pengujian lab dibangun murni menggunakan Go standard library tanpa perlu mengunduh package eksternal atau menjalankan Docker. Eksperimen lokal ini membuktikan seluruh semantik penegakan constraint secara deterministik, namun sistem produksi masih perlu mengonfigurasi strategi migrasi bertahap seperti ADD CONSTRAINT NOT VALID.

Deterministic local simulations validate distributed consistency principles without heavy infrastructure overhead.

README.md:1-25

Prinsip Inti: Database Adalah Benteng Terakhir Kebenaran

Aplikasi boleh berganti dan berkembang, namun integritas data harus abadi.

TIGA PILAR INTEGRITAS DATA ARSITEKTUR MODERN

Deklaratif di Titik Tulis

Letakkan aturan keabsahan data sedekat mungkin dengan media penyimpanan agar terlindungi secara atomik.

TIGA PILAR INTEGRITAS DATA ARSITEKTUR MODERN

Pemisahan Peran yang Jelas

Aplikasi melayani kenyamanan interaksi pengguna; database constraints menjamin kebenaran absolut.

TIGA PILAR INTEGRITAS DATA ARSITEKTUR MODERN

Tidak Otomatis Kebal

Memasang constraint tidak otomatis menyelesaikan aturan lintas tabel; pahami batas row-scoped versus triggers.

Aplikasi backend akan terus ditulis ulang, microservices akan terus bertambah, namun data di database akan bertahan melampaui siklus hidup kode tersebut. Menyerahkan integritas data hanya pada kode aplikasi adalah perjudian berisiko tinggi. Database constraints memastikan bahwa apa pun bug yang terjadi di lapisan aplikasi, integritas data penyimpanan tidak akan pernah ternoda.

Applications are ephemeral; persisted data is durable. Enforce invariant truth at the storage tier.

Jangan pernah percayakan integritas data hanya pada application code; database constraint adalah benteng terakhir kebenaran absolut.

Database constraints serve as the final, immutable bastion of data correctness across evolving system architectures.

Software Engineering Lab 27 – Database Constraints: Benteng Terakhir Integritas Data

internal/engine/engine.go • internal/store/store.go
internal/dberr/errors.go • internal/model/model.go
cmd/demo/main.go • internal/store/store_test.go

REPOSITORY

`github.com/lukman-ss/software-engineering-lab/tree/main/labs/27-database-constraints`

Lukman / lukman-ss

Commit: c317c1d • Status: APPROVED
Test passing: 8 test scenarios (NotNull, Check, Unique, FK, PartialIndex, ConcurrencySafe, ConcurrencyUnsafe, ErrorClassification)
Race detector: Clean (go test -race ./... PASS in 1.330s)
Zero external dependencies: 100% Go standard library.